VIA Project: Milestone 2

COM-480 – Data visualization

Maëlys Bühler
maelys.buhler@epfl.ch

Nima Dekhli
nima.dekhli@epfl.ch

Alexandre Maquet
alexandre.maquet@epfl.ch

April 17, 2026

Contents

1 Introduction **2**

2 Description of the visualizations **2**
2.1 Departure table 2
2.2 Stop selection 3
2.3 Clock 3
2.4 Swiss map 3
2.5 Distance evolution 4
2.6 Line type proportion 4

3 Project goals **4**
3.1 Minimal viable product 5
3.2 Nice to have 5
3.3 More challenging ideas 6

4 Required tools **6**
4.1 Technologies 6
4.2 Relevant lectures 7

1 Introduction

Via is an interactive visualization that puts the player in a train station in Switzerland, with the goal of traveling by train to another specific, randomly selected station in a day, without knowing where it is and how to go there. This document describes the visualizations, tools, and goals of this project.

2 Description of the visualizations

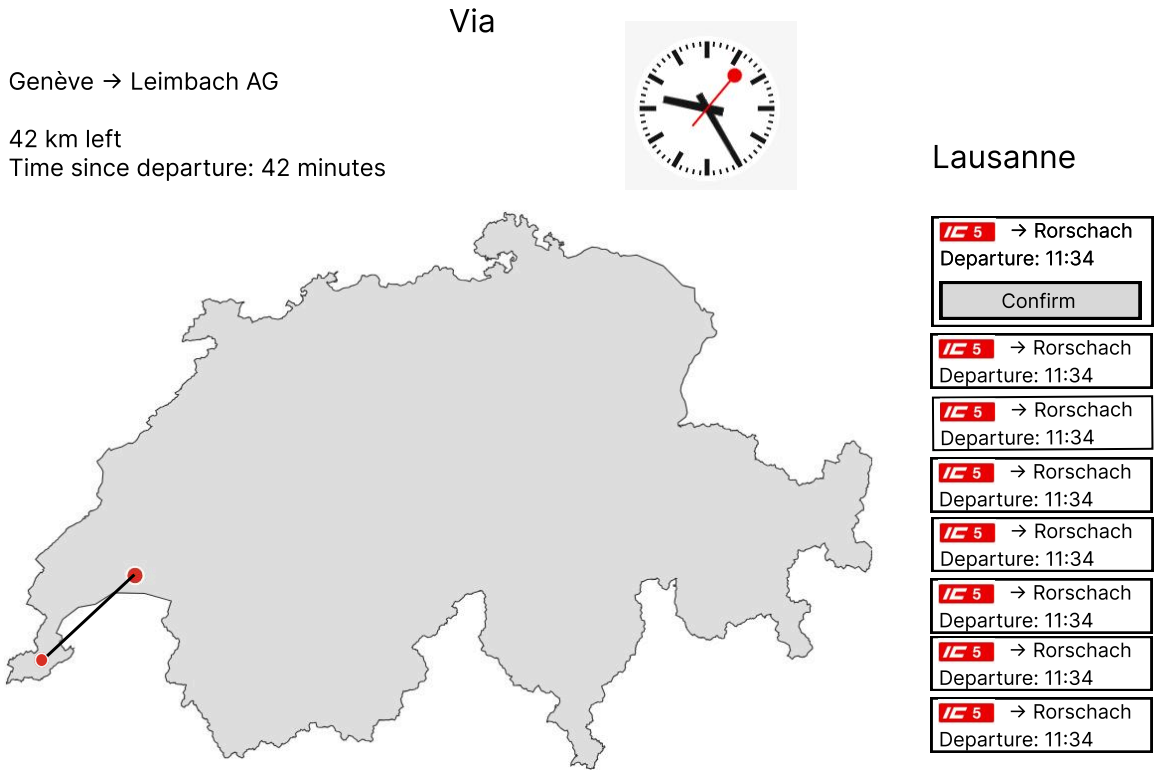

Figure 1: The main screen of the game, where we can select one train at a given station.

2.1 Departure table

At any time during the game, the player will be at a station. If the station is not the destination, then the game will display a timetable containing the next trains departing from the station. All subsequent departures that occur on the same day will be displayed, and the user can choose which one to take next. To do so, they click on the box that represents it and a *Confirm* button is shown to validate the choice.

The stops will each be in their own box on the right side of the page, in a column. The boxes representing the trains will have the type and number of the train (for example, IC 5), the departure time, and the destination of the train. The stops between the position and destination are not shown before the confirmation of which train is taken. This decision was made to avoid making the game too easy.

The sketch of this visualization can be seen in figure 1.

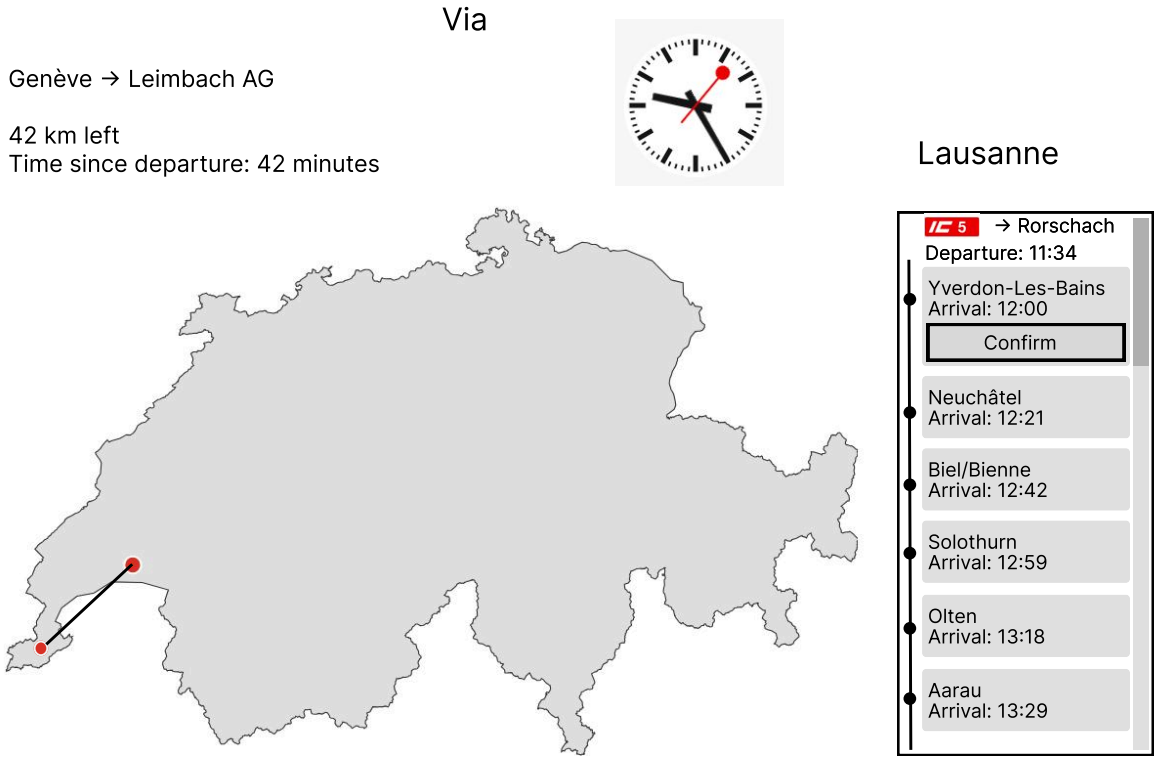

Figure 2: The main screen of the game, where we can select one particular stop to get off the train.

2.2 Stop selection

After confirming the train taken, we show each possible stop of the train. To do so, similarly to the departure table, each stop will be in a box. The information for each of them will include the name of the station and the arrival time. Similarly to the departures, clicking once on a stop displays the *Confirm* button, which we have to click to confirm the choice.

The sketch of this visualization can be seen in figure 2.

2.3 Clock

To represent the time in the simulation, a clock similar to that found in the SBB stations will be displayed on the screen, as can be seen in figures 1 and 3.

2.4 Swiss map

The map displays Switzerland. At each point in the game, the history of the trips taken so far is displayed to give the user an idea of the direction they are heading and where they are currently.

The sketch of this visualization can be seen in figures 1 and 2.

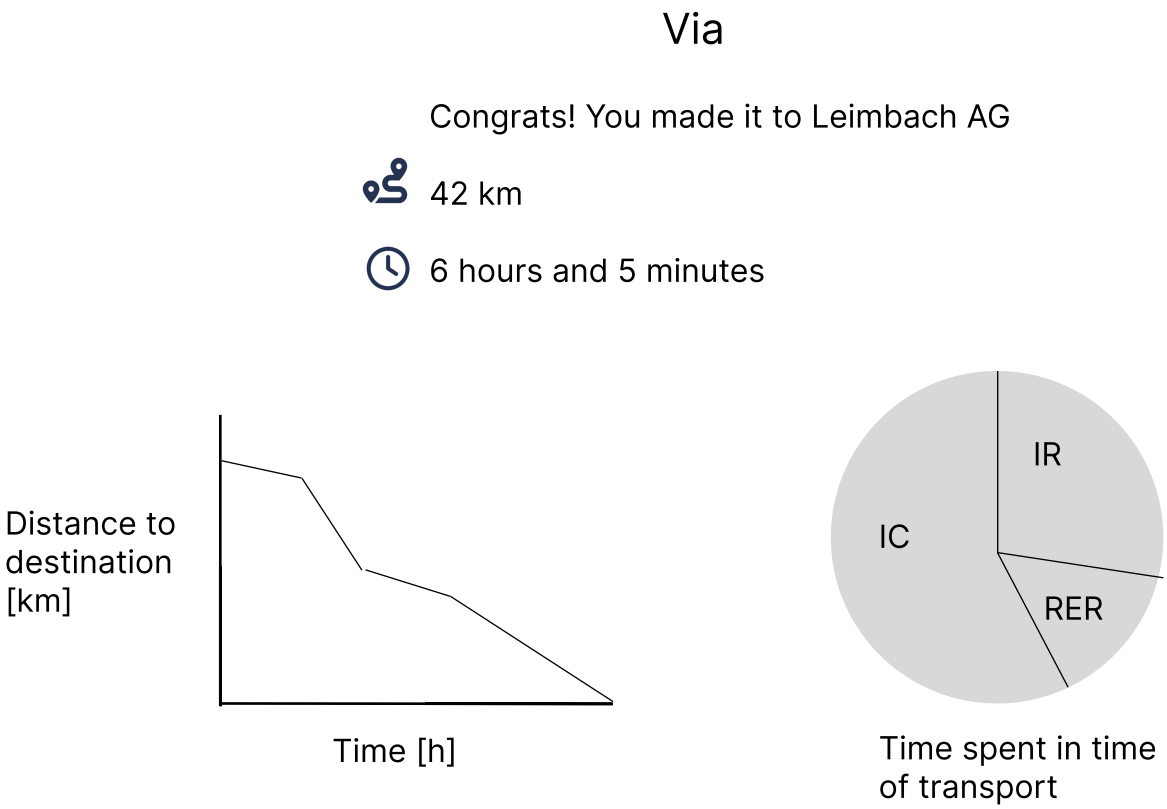

Figure 3: The end screen of the game

2.5 Distance evolution

To illustrate the changing distance between the player's position and the final station, we will use a graph plotting each visited station on the x-axis against the distance to the final destination on the y-axis (see figure 3). This type of visualization allows users to assess their route progression and overall performance in a glance, making it easy to quickly spot any navigational errors. This graph will be shown at the end of the game, after the user has found the final station.

2.6 Line type proportion

At the end of the game, alongside the distance evolution chart, we will display a pie chart showing the types of railway lines used during the journey (see figure 3). This visualization will help the user understand their predominant travel style: IC for long-distance trips across Switzerland, RER for local transit within a specific region, etc. We could also use another type of chart (e.g. bar chart) that shows the proportions of the line types that were taken.

3 Project goals

In this section, we describe the different goals we want to achieve in this project. We also distinguish between those who will be required to have a minimal viable product and those who are more creative or challenging.

3.1 Minimal viable product

We describe here the visualizations and the functionalities that are absolutely required to have a minimum viable product. Without these, the meaning of the project would be endangered and unsatisfactory.

Basic pre-processing The GTFS dataset contains a fairly large amount of details. Indeed, it allows us to exactly get the timetable for all transports in all Switzerland, for the whole year.

To make the game less tricky, we only allow trains and no other means of transport. Therefore, we can significantly reduce the size of the data set by removing all trips that do not correspond to that criterion.

One game plays over the duration of one day. Therefore, we can simply choose a given day of the year (preferably a weekday when there is no public holiday). This can also reduce the data set size (but not that much, since GTFS is smart enough to avoid useless repetitions).

In addition to filtering unused data, we have to find ways to segment the data into smaller pieces, so that the browser does not have to load the entire dataset each time, which could make the user experience less pleasant.

Departure table One of the main mechanics of the game is to be able to select a specific train in a given train station. This allows the user to navigate and therefore play the game. This is the reason why we consider it a required component. Without it, we cannot play the game.

List of stops When a player chooses a train, they have to decide where to go. To be able to do it, we need to be able to fetch and display all the stops of a given train route, and filter out the unreachable destinations (the ones before the current station). Without it, we cannot correctly play the game.

3.2 Nice to have

You can find here the ideas we have that we would like to implement if the time permits, but that could be dropped as necessary, without threatening the goal of the project.

Distance radius On the map, a circle around the current position is displayed with a radius being the distance to the destination, which means that the destination is on this circle. This could be displayed constantly, or on hover of any visited station. It allows the player to orient themselves and help them go in the correct direction.

Visual history timeline Using a visualization tool to display how much time the user spent in which train.

Exact path visualization Instead of drawing a straight line between two stations, drawing a realistic line that follows as much as possible the real path taken by the train. This makes the visualization much nicer to look at.

Random generator seeding Each user when entering the website will get a random seed, which is used as the input to the random generator, to determine the destination and the starting station. This seed will be given to the user so that they can share it with friends to allow them to play the same level as them. This means that it should also be possible to manually enter a seed in the interface.

Interactive map To make the experience more enjoyable and interactive, we can allow the user to zoom in and out of the map, and to explore more precisely. We could also automatically zoom the map depending on the current distance to the destination, which allows the user to see more precisely when necessary.

3.3 More challenging ideas

These ideas are more creative and challenging, but there is a high chance that we will not actually implement them.

Daily global seed Each day, generate a seed for a *Daily Challenge*. This allows all players to play on the same level. We can then display a leader-board with the scores of all participants. This functionality requires to implement an API server to generate the global seed and save the individual scores.

Elevation evolution chart We can create a chart that is displayed after the game representing the elevation of the visited train stations depending on the time of day. This would be slightly more challenging to implement, as we do not have the elevation of the stations in the datasets that we use, and would require us to either use an external API or find some additional dataset.

Spline color gradient based on distance We could display the spline of the trip using a color gradient, where the greener the color, the closer the point is to the final goal.

Animated trip history After the end of the game, we can display an animation that shows a train making the trip the user took on the map, with a speed proportional to the speed of the real connection.

4 Required tools

In this section, we describe the tools that we need to realize the project, alongside the corresponding course lectures.

4.1 Technologies

The website will use the Vue.js framework. This will allow us to create reusable and dynamic components. To create these components, we will use Typescript, as it offers more features than Javascript. The website will be hosted on Github Pages.

For the data visualization part, we will use a GTFS dataset preprocessed using a python script. Then we will use D3.js to generate multiple charts and the map. Specifically, we will combine the d3-geo module and GeoJSON data to render the map and plot the railway network.

4.2 Relevant lectures

The relevant lectures for the project are lectures 4.2 to learn about the basics of D3.js, 5.1 and 5.2 on interaction, 7.1 and 7.2 on designing visualization with D3 and what to be careful about when creating charts, 8.1 and 8.2 for the map, and GeoJSON to create and manipulate map objects to draw the railways network.